Ai Prompts

Ai tools Used : Antigravity, Claude (On a mix due to usage limit Constrain).

Prompt 1: Problem Statement

Please write a concise problem statement for a KYC onboarding platform that

* tracks onboarding evidence, validates completeness, schedules periodic reviews, generates expiry alerts, stores a full audit trail, and provides a public dashboard for multiple analyst personas.

Actors: Operations Analyst, Risk Reviewer, Compliance Officer, Data Scientist.

Compliance frameworks: FATF, FinCEN CDD, EU 5/6AMLD.

Prompt 2: Functional Requirements

List the functional requirements for the system : UI / Dashboard, REST API, Core pipeline logic, CLI, Testing & Validation, Documentation.

Prompt 3: Architecture Overview

Create a Mermaid flow-chart that shows the high-level components the

direction of data flow . Return the chart only.

Prompt 4: Data Model

Design a relational schema for the following entities (include PK, FK,

cardinality, required fields, enums, and validation rules):

Customer, CustomerType, KYCRequirement, Document, Verification,

RiskRating, BeneficialOwner, ReviewCase, Alert, AuditEvent, GroundTruth.

Use the naming conventions CUS-xxxxx, DOC-<cust>-001, … Provide:

a) `schema.sql`,

b) a markdown data dictionary,

c) a Mermaid `erDiagram`.

Prompt 5: Synthetic Data Generator

Write a deterministic Python script (`src/generator.py`) that, given seed

`20261022`, creates:

- 8 000 customers,

- 20 000 documents,

- 1 000 corporate entities,

- 3 000 review cases,

and then injects the five labelled edge-cases from the ground-truth spec.

Export the data as SQLite ,CSV and JSON.

Include a function generate(seed: int) -> None

Prompt 6: HTTP Server

Implement src/web_server.py using only the Python standard library.

Prompt 7: Pipeline Classes

Create `src/pipeline.py` with three classes:

- LiveOnboardingPipeline for onboard_new_customer, upload_and_process_evidence

- BatchSurveillancePipeline to run_daily_surveillance.

- TimeMachineEngine for advance_time

All methods must open a fresh SQLite connection, perform INSERTs only,

and insert an audit_events row for every state change.

Prompt 9: CLI Wrapper

Write src/cli.py that uses argparse to expose the following commands: `onboard`, `upload-doc`, `batch-surveillance`, `advance-time`, `resolve-review`. Each command should instantiate the appropriate pipeline class and print the

resulting dict as pretty-JSON.

Prompt 10: Tests

Provide a unit/integration test suite tests/ that:

1. Starts the server on a random free port,

2. Calls the `onboard` endpoint, then `upload-doc`,

3. Executes `batch-surveillance`,

4. Verifies KPI numbers (`total_customers`, `avg_completeness_pct`, etc.).

Use only the std-lib (`unittest`, `http.client`). Include a coverage command.

Prompt 11: Documentation

Generate a comprehensive `README.md` that contains:

Badges (Python, seed, precision/recall), Project overview, Quickstart , Architecture diagram , Directory layout License suggestion.